

Formatting Instructions for COLM 2026

Conference Submissions

Anonymous authors

Paper under double-blind review

Abstract

One important approach to improving the reasoning abilities of Large Language Models (LLMs) is to convert natural-language questions into logical propositions so that reasoning is performed via external tools such as logic solvers. However, translating ambiguous natural language into logical form is a known bottleneck that can introduce errors. Here, we study an LLM-based pipeline that, given a text first converts the text into a propositional knowledge base consisting of primitive propositions, their natural-language interpretations, and propositional formulas capturing the logical constraints expressed in the text. We then answer natural-language queries by translating them into a propositional formula over that knowledge base and solving it with a SAT solver. We study how different design choices affect end-to-end reasoning and translation accuracy. In particular, we study the impact of: (i) using OpenIE triples to support text-to-logic conversion; (ii) deterministic knowledge-base enrichment; (iii) shortcut-based query translation that can bypass some LLM calls; and (iv) query expansion with multiple variants and majority voting. Using a fractional factorial experimental design, we identify the effects of the different components and combinations. In particular, where additional complexity leads to diminishing returns.

1 Introduction

Reliable reasoning over text remains a core challenge in AI. Given a text, users need to determine whether a statement is entailed, contradicted, satisfiable, or completely irrelevant. Large language models (LLMs) provide natural-language interfaces for such tasks, but they conflate two fundamentally different problems: *language semantics* (extracting meaning from text) and *logical reasoning* (deriving valid conclusions from premises stated in the text). This conflation is the source of their unreliability. LLMs hallucinate facts, preferring external knowledge to the input document (Ji et al., 2023); miss information in long contexts, prioritizing the start and end while neglecting the middle (Liu et al., 2024a); and produce plausible but logically unsound conclusions (Huang & Chang, 2023).

A recent survey identifies neuro-symbolic methods, which separate language semantics from logical reasoning, as a promising paradigm for improving reasoning in LLMs (Yang et al., 2025). One class of neuro-symbolic methods translates natural language into formal representations, delegates inference to symbolic solvers, and interprets results—the LLM serves as a bridge between natural language and formal logic. However, these approaches are typically evaluated on benchmarks that favor the specific formalism they adopt, making cross-method comparison difficult. FOLIO (Han et al., 2024) and ProofWriter (Tafjord et al., 2021) are constructed around first-order logic; LogiQA2.0 (Liu et al., 2023) and LogicBench (Parmar et al., 2024) frame propositional and multi-step logical reasoning as multiple-choice questions; and DocNLI (Yin et al., 2021) and ContractNLI (Koreeda & Manning, 2021) measure textual entailment (instead of logical) over longer text.

To better understand the transition from natural language to logic, we study a configurable pipeline that converts text into a propositional knowledge base of atomic propositions along with propositional formulas representing hard constraints, and then answers queries via a SAT solver. Using four binary design factors, a full factorial experiment would require

45 $2^4 = 16$ runs per query—costly, especially when each run involves potentially multiple
 46 LLM calls. We adopt a 2_{IV}^{4-1} fractional factorial design (Box et al., 2005), which reduces the
 47 experimental runs to 8 while preserving the ability to estimate all main effects and most
 48 two-factor interactions. This allows us to identify the most influential pipeline components
 49 with statistical rigor.

50 We evaluate this pipeline on three datasets: LogicBench, a variation of DocNLI, and a
 51 custom hand-written “Alice” one. LogicBench tests reasoning over short premises, while
 52 DocNLI evaluates entailment over longer documents. Because DocNLI measures textual
 53 rather than logical entailment, we combine an LLM and human expertise to generate queries
 54 that require logical inference from the source text. These two datasets purposefully avoid
 55 modal language, focusing on classical propositional reasoning. To evaluate reasoning in the
 56 presence of modalities, we introduce the third set of questions, called Alice, which includes
 57 modal words such as “always,” “usually,” “rarely,” and “never.” Different configurations
 58 may favor different datasets, and our factorial analysis identifies how the components
 59 behave across this diverse evaluation suite.

60 2 Related works

61 2.1 Neuro-symbolic solvers

62 Recent work combines LLMs with symbolic solvers to improve logical reasoning. Logic-
 63 LM (Pan et al., 2023) introduced a framework that uses LLMs to translate natural language
 64 text and queries into a first-order logic (FOL) knowledge base, iteratively solve FOL syntax
 65 issues refining from error messages, and offloads to an SMT solver to determine entailment.
 66 Logic-LM++ (Kirtania et al., 2024) extends this with multi-step refinement using pairwise
 67 comparisons to evaluate candidate revisions, achieving 78.9% on FOLIO and 79.7% on
 68 ProofWriter. LINC (Olausson et al., 2023) takes a similar approach but uses the Prover9
 69 theorem prover for deductive inference and, instead of fixing incorrect FOL syntax, LINC
 70 samples (at default 10) candidate translations and uses a majority voting strategy. LINC
 71 reaches 87% on ProofWriter with GPT-4, outperforming chain-of-thought prompting by
 72 10–38% absolute depending on model size.

73 Both systems formalize problems per query on short benchmarks; in contrast, our approach
 74 studies the effect of first converting the initial document into a set of logical propositions and
 75 constraints, a so-called *logification*, of the document. Then, it supports repeated querying
 76 over longer texts. A major source of error, as noted in the cited works, arises from the
 77 FOL formulation. FOL translation requires choosing predicate arities, variable scopes,
 78 and quantifier structures—decisions that introduce semantic ambiguity beyond the LLM’s
 79 competence. Logic-LM’s execution rate drops from 99% (propositional-like ProofWriter) to
 80 79.9% (FOL-based FOLIO). And LINC’s error analysis attributes failures primarily to the
 81 FOL translation process—implicit information loss and representation choices—rather than
 82 the symbolic reasoning itself. We therefore deliberately restrict ourselves to propositional
 83 logic.

84 Our approach also fits within a broader line of work that augments LLMs with external
 85 tools to offload computational reasoning. For example, PAL (Gao et al., 2023) and Program-
 86 of-Thought (Chen et al., 2023) generate Python code for arithmetic reasoning, delegating
 87 calculation to an interpreter while the LLM handles problem decomposition. SatLM (Ye et al.,
 88 2023) uses declarative prompting to cast problems as satisfiability queries, achieving state-
 89 of-the-art on LSAT and BoardgameQA by offloading inference to an automated theorem
 90 prover. Logic-of-Thought (Liu et al., 2024b) injects propositional logic into prompts as
 91 an augmentation layer: it extracts logical expressions, extends them via inference rules,
 92 translates them back to natural language, and appends this enriched context to the original
 93 prompt for the LLM to reason over. All of these approaches outperform CoT baselines
 94 by separating what LLMs do well (processing language) from what they do poorly (valid,
 95 intricate reasoning).

2.2 Semantic similarity and textual entailment

Our pipeline leverages pretrained models for semantic retrieval and entailment scoring. Sentence-BERT (SBERT) (Reimers & Gurevych, 2019) modifies the BERT architecture with siamese networks to produce fixed-size sentence embeddings that capture semantic similarity; cosine similarity between embeddings enables efficient retrieval without cross-encoding every pair. For textual entailment, we use a DeBERTa-v3-large cross-encoder fine-tuned on NLI datasets (He et al., 2021). Unlike SBERT’s bi-encoder, a cross-encoder jointly encodes a premise-hypothesis pair and outputs probabilities over entailment, contradiction, and neutral. While more accurate, it is computationally expensive so we use SBERT for retrieval and cross-encoder NLI for scoring.

For syntactic analysis and lexical knowledge, we rely on established NLP tools. SpaCy (Honnibal et al., 2020) provides tokenization, part-of-speech tagging, and dependency parsing, which we use for negation detection and grammatical analysis. WordNet (Miller, 1995) supplies synonym sets and antonym relations, enabling our modal-opposite detection and query expansion mechanisms. Stanford CoreNLP’s OpenIE (Angeli et al., 2015), accessed via Stanza (Qi et al., 2020), extracts subject-predicate-object triples from text.

3 System architecture

Our pipeline follows a modular design with three components: (1) `from_text_to_logic`, which *logifies* text into atomic propositions, their translation into natural language, and hard constraints inferred from the text; (2) `interface_with_user`, which handles translation from natural language queries to propositional formulas over the logified atomic propositions; and (3) `logic_solver`, which performs reasoning via a SAT solver. This modularity facilitates our fractional analysis with four boolean factors, two factors for enriching the logification and the other two for enriching queries.

3.1 From text to logic

We implement the extraction of natural language text into a logified structure in up to four stages. As part of our factorial experiment, this extraction depends on two binary factors described in Table 1: `use_openie`, which enriches the original text with subject-predicate-object triples, and `use_enrichment_kb` which enriches the original logified file to better support both modal questions and related propositional queries.

Text-to-knowledge-base construction stage		
	Factor	Description
T1	<code>use_openie</code>	Supplies Stanford CoreNLP OpenIE relation triples as auxiliary input to the LLM during text-to-logic conversion.
T2	<code>use_enrichment_kb</code>	Applies enrichment (modal-pair verification, negation detection, finite-domain auxiliaries, and conflict resolution) to expand and validate the knowledge base.

Table 1: Binary factors for the text-to-logic stage of our factorial experiment. Each factor enables or disables a distinct pipeline component.

3.1.1 Relation triple extraction

We use Stanford CoreNLP’s OpenIE system to extract subject-predicate-object triples from the text. We first apply native Stanza coreference resolution to ensure that triples consistently refer to canonical entities rather than pronouns, e.g., replacing “they” with “Alice”. There are cases where the grammatical structure fails for OpenIE, e.g. “Alice sleeps soundly” yields no OpenIE triple, so we extract the subject-verb pair from Stanza’s dependency parse.

3.1.2 LLM logic conversion

We input the original text T and, when `use_openie` is enabled, its extracted triples to GPT-5.2 (with medium reasoning effort) using a few-shot Chain-of-Thought (CoT) prompt. The LLM identifies atomic propositions $\mathcal{P}(T) := \{P_1, P_2, \dots, P_n\}$ and a collection of propositional formulas over $\mathcal{P}(T)$ representing constraints $\mathcal{C}(T) := \{C_1, C_2, \dots, C_m\}$. For each constraint, the LLM assigns an initial weight $w \in [0, 1]$ reflecting confidence that the constraint is obligatory; for example, if $w_1 = 1$ then the LLM determined that the constraint C_1 is enforced from the text. Note, we use two different prompts according to whether we are testing the Alice dataset or not. Only in the Alice dataset do the atomic propositions contain modal expressions representing degrees of uncertainty (e.g., “must,” “likely,” “may”). For the other datasets, the LLM strips all modality away from the atomic propositions, and the assigned weight reflects this degree.

The LLM generates an initial JSON structure containing: (i) primitive propositions with their natural-language translations and textual evidence, and (ii) constraints with their logical formulas, translations, evidence, and initial weights (see Appendix A.1 for the full prompt and Appendix ?? for a worked example).

3.1.3 Enriching the knowledge base

When `use_enrichment_kb` is enabled, we apply four deterministic enrichment steps to the initial logified structure:

1. **Modal-pair verification:** For each atomic proposition containing a modal word (e.g., “always,” “usually,” “rarely”), we add another atomic proposition, if missing, that removes the modality. For example, if “*Alice rarely studies late at night*” is an atomic proposition in the original logified JSON, we insert a new one “*Alice studies late at night.*” In particular, this step only applies to the Alice dataset because for the other datasets, the modalities are already stripped.
2. **Modal-opposite detection:** Using WordNet antonyms, we identify propositions with opposing modal words (e.g., “typically” vs. “rarely”) over the same base event. When such pairs are found, we add mutual exclusion constraints. For instance, if the text contained both P_i : “*Alice typically studies hard*” and P_j : “*Alice rarely studies hard,*” we would add $P_i \Rightarrow \neg P_j$ and $P_j \Rightarrow \neg P_i$ to enforce that these cannot both hold.
3. **Negation detection:** We use SpaCy to detect whether the original logified knowledge base includes atomic propositions that are negations (e.g., “not,” “never,” “doesn’t”). If present then we extract the affirmative clause, and add it as a new atomic proposition if its not already there determined with SBERT.
4. **Finite-domain auxiliaries:** Propositional logic lacks the expressivity to represent finite-domain constraints directly (e.g., “*Alice studies exactly one major*”). To partially address this, we detect propositions involving values from known finite domains (e.g., days of the week) and generate auxiliary propositions for alternative values, adding mutual exclusion constraints between them.

Finally, we apply NLI-based conflict resolution to detect and resolve cases of inconsistency in the resulting logified knowledge base. If a proposition and its negation are asserted as constraints, we keep the proposition with higher NLI entailment score from the source text.

3.1.4 Final logified structure

We determine which weighted constraints to keep as hard constraints for the final logified JSON. For each constraint C_i with LLM-assigned weight w_i^{llm} , we compute an NLI entailment score w_i^{nli} as follows. We retrieve the top-10 most similar propositions using SBERT and measure the maximum entailment probability using a cross-encoder NLI model. The combined score is the product $w_i = w_i^{llm} w_i^{nli}$. Constraints which exceed a fixed hardness threshold ($w_i \geq 0.9$ in our experiments) become the final hard constraints; the remaining become soft constraints. Entries in the final JSON have a template shown in Appendix A.2.

3.2 Interface with user

Once we have logified the text, we can process queries about that text and solve them with a logic solver. The purpose of this module is to translate a given query into a propositional formula. Translation depends on two binary factors described in Table 2: `use_shortcuts`, which avoids some cases of calling an LLM, and `expand_query`, which generates synonymous queries and uses a voting strategy for translation.

Query-to-formula translation stage		
	Factor	Description
Q1	<code>use_shortcuts</code>	Activates deterministic shortcut detectors (modal opposites, lexical antonyms, and implication contradictions) that bypass the LLM when certain patterns are found in the query.
Q2	<code>expand_query</code>	Creates query alternatives using WordNet synonym variants filtered by SBERT similarity, and enables majority-vote ensembling over multiple LLM calls to select the final logical formula. Both mechanisms are activated jointly.

Table 2: Binary factors for the query-to-formula translation stage of our factorial experiment.

3.2.1 Query translation

Given a natural-language query and the logified knowledge base, we first determine the query type. If the query is phrased as a question, we convert it to a declarative statement—for example, “Does Alice study computer science?” becomes “Alice studies computer science.” We then retrieve the top-10 closest propositions to this statement using cosine similarity over SBERT embeddings. This retrieved subset is provided to the LLM (see Appendix A.4 for the prompt), which outputs either a propositional formula over the retrieved propositions, or NONE if the query is unrelated to the source text.

3.2.2 Shortcut detectors

When `use_shortcuts` is enabled, deterministic shortcut detectors can resolve certain queries before invoking the LLM. These include detecting contradictions between modal expressions such as “must” and “must not,” identifying direct lexical antonyms using WordNet (for example, “always” versus “never”), and recognizing implication-based contradictions such as “X requires Y” versus “X does not require Y.” When a shortcut fires, it bypasses LLM translation entirely, reducing latency and avoiding potential translation errors.

3.2.3 Query expansion and voting

When `expand_query` is enabled, we apply two mechanisms to improve translation robustness. First, synonym expansion generates multiple queries using WordNet synonyms filtered by SBERT similarity to ensure semantic fidelity; each variant is translated independently by the LLM. Second, majority voting selects the final formula by vote over the multiple candidate translations, weighted by translation confidence. This ensemble approach reduces sensitivity to specific query phrasings.

3.3 Logic solver

We use PySAT’s RC2 solver (Ignatiev et al., 2019) to determine results. To check entailment, we add $\neg Q$ as a hard constraint and test satisfiability; if the resulting formula is unsatisfiable, then the knowledge base entails Q . To check contradiction, we instead add Q as a hard constraint; if this makes the formula unsatisfiable, then the knowledge base entails $\neg Q$. To check uncertainty, we verify that neither case holds—that is, both Q and $\neg Q$ are

consistent with the knowledge base. For queries where the translation returns NONE, we report NOT_MENTIONED: the query concerns content outside the knowledge base’s scope.

4 Experiments

4.1 Datasets

4.1.1 LogicBench

(Parmar et al., 2024) is a comprehensive logical reasoning benchmark comprising 25 distinct inference patterns across propositional, first-order, and non-monotonic logics. We use the LogicBench(Eval) subset, in particular binary question-answer instances. Each example presents a natural language context encoding logical premises and asks whether a conclusion follows. The dataset tests fundamental inference rules including modus ponens, modus tollens, disjunctive syllogism, hypothetical syllogism, and constructive/destructive dilemmas. We evaluate on 160 examples balanced across 8 propositional logic patterns.

4.1.2 DocNLI

(Yin et al., 2021) is a document-level natural language inference dataset containing premise-hypothesis pairs from diverse sources, including news articles, Wikipedia passages, and other document corpora, labeled for entailment vs. not entailment. Because these queries target textual rather than strict logical entailment, we regenerate them to better exercise our neuro-symbolic pipeline. We prompt an LLM (see Appendix A.3) to generate 16 single-sentence query per text: 4 *entailment*, 4 *contradiction*, 4 *uncertain*, and 4 *not mentioned*. This yields 160 queries across 10 texts, evenly distributed among the four output categories. The new queries are then evaluated by the authors to detect any imprecision.

4.1.3 Alice

“Alice” is a controlled dataset we constructed to evaluate reasoning capabilities in everyday scenarios. These data consist of a short document describing “Alice,” a university student, with 20 hypothesis statements spanning four inference types: direct entailment, contradiction, neither entailed nor contradicted, and statements not mentioned in the source. Unlike the other works, this dataset supports reasoning over modal language such as “always,” “usually,” “rarely,” and “never,” allowing us to test the effect of uncertainty on translation.

4.2 Experimental approach: fractional design

Run	Q_1	Q_2	T_1	T_2
1	—	—	—	—
2	+	—	—	+
3	—	+	—	+
4	+	+	—	—
5	—	—	+	+
6	+	—	+	—
7	—	+	+	—
8	+	+	+	+

Table 3: 2^{4-1}_{IV} fractional factorial design with generator $T_2 = Q_1 Q_2 T_1$.

We recall that fractional factorial designs are a class of experimental designs that enable efficient exploration of multiple factors using only a subset of all possible combinations.

We evaluate our pipeline using four binary factors: Two factors affecting the text-to-logic conversion (T_1 : use_openie, T_2 : use_enrichment_kb; see Table 1) and the other two factors involved on the query translation (Q_1 : use_shortcuts, Q_2 : expand_query; see Table 2). We

250 adopt a 2_{IV}^{4-1} fractional factorial design with generator $T_2 = Q_1 Q_2 T_1$, yielding 8 runs per
 251 query summarized in Table 3.

252 4.3 Results

253 For our modification of DocNLI, Figure 1 shows that overall accuracy varies only modestly
 254 across runs, remaining roughly in the 0.43–0.51 range. This suggests that no single configura-
 255 tion clearly dominates end-to-end performance on DocNLI. The class-wise breakdown,
 256 however, reveals larger differences: *uncertain* and *not mentioned* are consistently easier than
 257 *entailment* and *contradiction*. This pattern indicates that the pipeline is better calibrated for
 258 recognizing non-entailing cases than for recovering logically supported or contradicted
 259 statements.

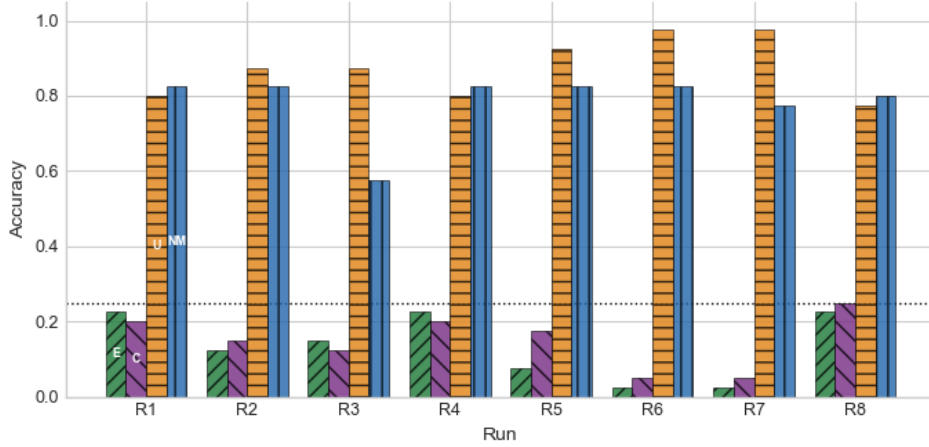


Figure 1: Per-category classification accuracy on the DocNLI edited dataset across the eight runs of the 2_{IV}^{4-1} fractional factorial design. Within each run group, bars represent from left to right: Entailment (green), Contradiction (purple), Uncertain (orange), and Not Mentioned (blue); category abbreviations are labelled on run R1. The dotted line marks the 25 % random-chance baseline.

260 Figure 2 summarizes the estimated main effects of the four binary factors by category. The
 261 strongest pattern is that enabling OpenIE (T_1) improves performance on the *uncertain* class,
 262 but reduces accuracy on *entailment* and *contradiction*. Query expansion and voting (Q_2)
 263 helps *entailment* and *contradiction*, but harms *not mentioned*, suggesting that paraphrase
 264 expansion can increase false alignment with the source document. By contrast, shortcut-
 265 based translation (Q_1) and knowledge-base enrichment (T_2) have smaller and more mixed
 266 effects. Overall, these results show that the design choices do not act uniformly across
 267 inference types: a component that improves one category may degrade another.

268 For LogicBench, Figure 3 shows the main effects of the four factors separated by label.
 269 The effects are modest overall, but they differ by class: for entailment examples, *Shortcuts*,
 270 *Expand*, and *Enrich* yield small positive effects, while *OpenIE* has a slight negative effect.
 271 For not-entailment examples, *Enrich* has the largest negative effect, followed by *Expand* and
 272 *Shortcuts*, while *OpenIE* remains near zero.

273 Our analysis of 320 questions from LogicBench across all 8 configurations reveals a sur-
 274 prising and counter-intuitive pattern: SBERT confidence is negatively correlated with
 275 prediction accuracy ($r = -0.229$, $p < 0.0001$). In other words, the system is systematically
 276 more confident when it makes incorrect predictions. The effect is most pronounced for
 277 missed entailments, where the system predicts *Uncertain* with the highest average SBERT
 278 confidence (0.889) despite the ground-truth label being *Yes*. Our results suggest that the
 279 translation step often produces formulas that superficially resemble the query—as reflected
 280 in high back-translation similarity—but fail to capture the semantic relationship required
 281 for entailment.

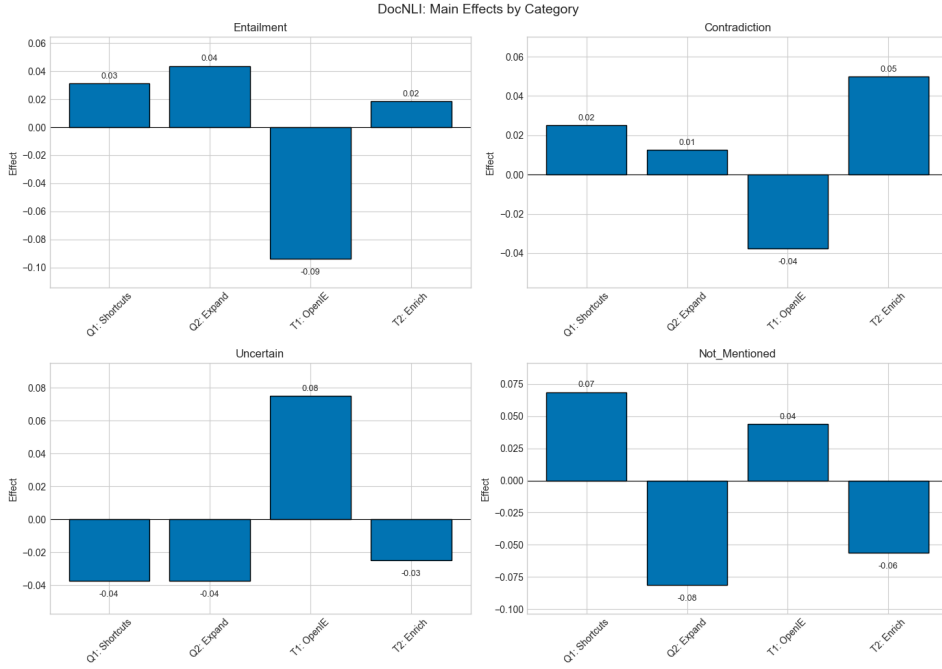


Figure 2: Estimated main effects of the four binary design factors on DocNLI, broken down by category. Positive values indicate that enabling the factor improves accuracy.

Similarly, we found that NLI confidence shows no significant correlation with accuracy. Predictions with high contradiction confidence ($NLI < -0.6$) achieve 90.5% accuracy, whereas predictions in the neutral zone ($-0.2 < NLI < 0.2$) achieve only 24.8% accuracy. This bimodal pattern suggests that the NLI score could support selective prediction: the system can be trusted when it confidently detects contradiction, but should abstain or flag cases for review when the NLI score falls in the neutral zone. The 117 questions in this neutral region account for a disproportionate share of the errors, indicating that uncertainty in the entailment relation—rather than high confidence in the wrong direction—is the dominant failure mode.

For the Alice dataset, standard neural similarity metrics (SBERT, NLI) likewise fail to predict translation quality in a neuro-symbolic pipeline. In particular, SBERT confidence is negatively correlated with accuracy ($r = -0.23, p < 0.001$), indicating that surface-level semantic similarity is insufficient for validating logic translations. More broadly, NLI is limited here because it evaluates entailment between natural-language strings rather than between the translated logical formula and the knowledge base, and therefore has no access to whether the SAT solver will ultimately find the formula satisfiable. This highlights the need for logic-aware verification methods.

5 Conclusion

We present a configurable neuro-symbolic pipeline that converts natural language text into a propositional knowledge base and answers queries via SAT solving. Using a fractional factorial design, we conclude that the effects of individual pipeline components are not uniform: on DocNLI, different design choices improve different inference categories, and no single factor consistently improves performance across all of them. An example from LogicBench in Appendix A.5, illustrates that some apparent errors may stem not from faulty reasoning, but from differences in interpretation between our pipeline and the datasets. Namely, how results depend on how strictly benchmarks adhere to the source text.

Limitations. Our adoption of propositional logic restricts expressivity. First-order constructs such as universal and existential quantification must be flattened into finite conjunctions or disjunctions over enumerated entities. At best inflates the number of atomic propositions, and at worse fails when domains are open-ended. By using SAT rather than SMT solvers, we lack native support for arithmetic and numerical reasoning. Statements involving comparisons, arithmetic relationships, or ordering is not easily captured.

Further work. Our framework already supports infrastructure for many-valued and modal logics beyond classical propositional logic. Extending the pipeline to leverage these capabilities would enable more faithful representation of the text we logify. We would be interested in comparing the accuracy of logifying text over those alternative logics. Additionally, we can modify our logification prompt to process much larger documents. Our current approach could scale to texts that exceed LLM context windows. The logified knowledge base, once constructed, supports efficient querying regardless of the original document length—enabling reasoning over arbitrarily large corpora.

Acknowledgments

We thank the Sundial team for their invaluable support and collaboration throughout this project.

Ethics Statement

This paper investigates AI-assisted formal reasoning over natural-language text. While neuro-symbolic pipelines can improve reliability by delegating inference to symbolic solvers, they remain vulnerable to errors in language understanding, including ambiguity, omission, and misformalization. As a result, we do not view the system described here as appropriate for fully autonomous use in high-stakes domains without human oversight.

We also disclose that AI tools were used during this project. In particular, we used the Sundial agent to assist in developing tests for the code and repository, refining implementation details, and providing feedback on parts of the manuscript. All writing was done by the authors, who take full responsibility for the final content of the paper and code.

References

- Gabor Angeli, Melvin Jose Johnson Premkumar, and Christopher D. Manning. Leveraging linguistic structure for open domain information extraction. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 344–354, 2015.
- George E. P. Box, J. Stuart Hunter, and William G. Hunter. *Statistics for Experimenters: Design, Innovation, and Discovery*. Wiley, Hoboken, NJ, 2nd edition, 2005.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 10764–10799. PMLR, 2023.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Bhattacharya, Bo Pang, Chenhao Tan, Dian Yu, and Hong Wang. FOLIO: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 22017–22031, 2024.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*, 2021.

- 355 Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy:
356 Industrial-strength natural language processing in python, 2020.
- 357 Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A
358 survey. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 1049–1065,
359 2023.
- 360 Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. RC2: An efficient MaxSAT
361 solver. *Journal on Satisfiability, Boolean Modeling and Computation*, 11(1):53–64, 2019.
- 362 Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin
363 Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language
364 generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- 365 Shashank Kirtania, Priyanshu Gupta, and Arjun Radhakrishna. LOGIC-LM++: Multi-step re-
366 finement for symbolic formulations. In *Proceedings of the 2nd Workshop on Natural Language
367 Reasoning and Structured Explanations (NLRSE)*, pp. 54–63. Association for Computational
368 Linguistics, 2024.
- 369 Yuta Koreeda and Christopher Manning. ContractNLI: A dataset for document-level natural
370 language inference for contracts. In *Findings of the Association for Computational Linguistics:
371 EMNLP 2021*, pp. 1907–1919, 2021.
- 372 Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue
373 Zhang. LogiQA 2.0—an improved dataset for logical reasoning in natural language
374 understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:
375 2947–2962, 2023.
- 376 Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni,
377 and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions
378 of the Association for Computational Linguistics*, 12:157–173, 2024a.
- 379 Tongxuan Liu et al. Logic-of-thought: Injecting logic into contexts for full reasoning in large
380 language models. *arXiv preprint arXiv:2409.17539*, 2024b.
- 381 George A. Miller. Wordnet: a lexical database for english. *Communications of the ACM*, 38
382 (11):39–41, 1995.
- 383 Theo X. Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua B.
384 Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning
385 by combining language models with first-order logic provers. In *Proceedings of the 2023
386 Conference on Empirical Methods in Natural Language Processing*, pp. 5153–5176, 2023.
- 387 Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empower-
388 ing large language models with symbolic solvers for faithful logical reasoning. In *Findings
389 of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824, 2023.
- 390 Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh
391 Mashetty, Arindam Mitra, and Chitta Baral. LogicBench: Towards systematic evaluation
392 of logical reasoning ability of large language models. In *Proceedings of the 62nd Annual
393 Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13679–
394 13707, 2024.
- 395 Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. Stanza:
396 A Python natural language processing toolkit for many human languages. In *Proceed-
397 ings of the 58th Annual Meeting of the Association for Computational Linguistics: System
398 Demonstrations*, pp. 101–108, 2020.
- 399 Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese
400 BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural
401 Language Processing and the 9th International Joint Conference on Natural Language Processing
402 (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.

- 403 Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications,
404 proofs, and abductive statements over natural language. In *Findings of the Association for*
405 *Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634, 2021.
- 406 Xiao-Wen Yang, Jie-Jing Shao, Lan-Zhe Guo, Bo-Wen Zhang, Zhi Zhou, Lin-Han Jia, Wang-
407 Zhou Dai, and Yu-Feng Li. Neuro-symbolic artificial intelligence: Towards improving the
408 reasoning abilities of large language models. In *Proceedings of the Thirty-Fourth International*
409 *Joint Conference on Artificial Intelligence, IJCAI-25*, pp. 10770–10778, 2025. doi: 10.24963/
410 ijcai.2025/1195. Survey Track.
- 411 Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. SatLM: Satisfiability-aided language
412 models using declarative prompting. In *Advances in Neural Information Processing Systems*,
413 volume 36, pp. 45584–45607, 2023.
- 414 Wenpeng Yin, Dragomir Radev, and Caiming Xiong. DocNLI: A large-scale dataset for
415 document-level natural language inference. In *Findings of the Association for Computational*
416 *Linguistics: ACL-IJCNLP 2021*, pp. 4913–4922, 2021.

A Appendix

A.1 Logification Prompt

We use two logification prompts for converting natural language text into propositional logic. When not evaluating on the Alice dataset, we run the following few-shot, CoT prompt:

ROLE
You are a neuro-symbolic reasoning assistant specializing in translating natural language to zeroth-order propositional logic, aided by preprocessed OpenIE relation triples.

INTENDED TASK
Given natural language text T and POSSIBLY its Stanford OpenIE relation triples (you may NOT be given triples), extract:

1. PRIMITIVE PROPOSITIONS (P₁, P₂, ..., P_n)
 - Primitive, truth-evaluable statements that are not themselves connectives of finer propositions (e.g. no negated statements).
 - Non-redundant: avoid paraphrase duplicates; do not split into overlapping variants unless required by uncertainty rules
 - Include natural language meaning and textual evidence
 - Propositions must be specific, complete, and exhaustive for all constraints
 - CRITICAL: The proposition TEXT must be a simple declarative clause with no conditional words ("if", "when", "unless", "whenever", "not") or logical connectives ("and", "or").
2. CONSTRAINTS WITH WEIGHTS (C₁, C₂, ..., C_m)
 - Zero-order propositional formulas over {P₁, ..., P_n} suggested by the text
 - Each constraint has a llm_weight between 0 and 1 describing how strongly the text implies the formula holds
 - llm_weight = 1.0 means the constraint must hold
 - llm_weight = 0.5 means the constraint may be satisfied with 50% uncertainty
 - Include meaning, evidence, and reasoning from text
 - If contradictions exist, prioritize text intent; only include inconsistencies if genuinely intended
 - IMPORTANT: You must look at the primitive propositions and examine their translations. There may be common-sense semantic relationships between them. You must include constraints corresponding to these relationships with appropriate weights, including implied logical constraints.

METHODOLOGY

Let's work step-by-step as follows:

STEP 0: Review Exemplars

- Study examples for task intent and baseline quality expectations
- Use for guidance on ambiguous or interpretable text

STEP 1: Analyze Relation Triples

- If present, review triples to understand extracted entities and relationships
- CRITICAL: Triples are incomplete preprocessing hints. Follow this reconciliation protocol:
 1. Extract propositions from the original text first
 2. Use triples to recover missed arguments or entities
 3. Reject triples that introduce entities, relations, or facts absent from or contradictory to the text
 4. When triples conflict with text, always prioritize the text

STEP 2: Construct Primitive Propositions

```
478 - Extract atomic, truth-evaluable statements from text that cannot
479 further be broken down
480 - Ensure propositions are non-redundant and that they exhaust all
481 constraints
482 - Break down to genuinely atomic statements (no hidden logical
483 connectives)
484 - CRITICAL RULE: If text contains conditional or connective words
485 ("if", "when", "unless", "and", "or", "not"), extract component
486 clauses separately
487 - Example: "When Alice is focused, she completes her assignments on
488 time"
489   -> P_13: "Alice is focused" (simple declarative)
490   -> P_14: "Alice completes her assignments on time"
491         (simple declarative)
492   -> Then create constraint: C: P_13 ==> P_14 with weight 1.0
493 - Remove ALL modal/uncertainty words and use them to help inform the
494 weights of constraints: these are words indicating modality,
495 frequency, or strength such as always, never, usually, typically,
496 generally, often, frequently, sometimes, occasionally, rarely,
497 seldom, may, might, could, can, should, ought, encouraged,
498 discouraged, recommended, optional.
499
500 GUIDELINES:
501 - Use zeroth-order logic only (no quantifiers)
502 - Use consistent terminology: "primitive propositions" (matching JSON
503 field `primitive_props`)
504 - Flatten first-order-like statements: "Every student studies Tuesday"
505   -> identify all students (Alice, Bob) -> "Alice studies Tuesday /\ Bob
506 studies Tuesday"
507 - Ensure faithfulness to original text
508 - Extract implicit assumptions using text intent and common sense
509 - When triples conflict with text, prioritize text
510 - Constraints inform the primitive propositions, so update primitive
511 propositions when necessary
512 - Make sure to compare with the examples
513
514 GRAMMAR
515 Use standard zeroth-order propositional logic:
516 - Variables: P_1, P_2, ...
517 - Operators: not, and, or, implies, iff
518 - Use parentheses when needed to avoid ambiguity
519
520 OUTPUT
521 Output ONLY valid JSON following this schema:
522
523 {
524   "primitive_props": [
525     {
526       "id": "P_1",
527       "translation": <proposition meaning in natural language>,
528       "evidence": <location in text where proposition first appears;
529                 sentence indices are 0-based (first sentence is
530                 Sentence 0)>,
531       "explanation": <brief explanation for why this is atomic and
532                     cannot be broken down>
533     }, ...
534   ],
535   "constraints": [
536     {
537       "id": "C_1",
538       "formula": <propositional formula over P_i>,
539       "translation": <natural language translation; prefer brief direct
540                     quote or paraphrase>,
541       "llm_weight": <float 0.0-1.0>,
542       "evidence": <location in text; sentence indices are 0-based>,
```

```

543     "reasoning": <why this constraint has this weight>
544   }, ...
545 ]
546 }
547
548 If saving to disk, name the file "logified.json".
549
550 INPUT FORMAT
551 You will receive the original text T and OpenIE relation triples.
552 Sentence indices in triples are 0-based.
553
554 ORIGINAL TEXT:
555 <<<
556 [Original natural language text]
557 >>>
558
559 RELATION TRIPLES:
560 <<<
561 [JSON array where each triple is:
562  [subject, predicate, object, sentence_index]]
563 Example:
564 [["Alice", "likes", "Bob", 0], ["Bob", "studies", "hard", 1]]
565 Sentence indices are 0-based (first sentence is index 0).
566 >>>

```

567 The prompt is followed by two examples (omitted for brevity). For the Alice dataset we run
568 the following one-shot prompt to retain the modal wording.

```

569 ROLE
570 You are a neuro-symbolic assistant translating natural language to
571 propositional logic.
572
573 TASK
574 Given text T and optional OpenIE triples which attempt to support T,
575 extract:
576 1. PRIMITIVE PROPOSITIONS (P_1, P_2, ...) - atomic, truth-evaluable
577    statements
578 2. CONSTRAINTS (C_1, C_2, ...) - propositional formulas with
579    llm_weight (0-1)
580
581 RULES
582 - Propositions must be simple declaratives that are atomic; they must
583   not have connectives in them (e.g. they cannot be conditionals or be
584   negations).
585 - Extract conditionals as separate propositions + implication
586   constraint
587 - Modal words (may, should, typically, sometimes, rarely): create TWO
588   propositions:
589   * P_modal: original with modal word
590   * P_base: without modal word
591   Then add constraints: P_modal@1.0, P_base@weight,
592   P_base==>P_modal@1.0
593
594 ID FORMAT (IMPORTANT)
595 - Proposition IDs must be strictly P_N where N is a number
596   (e.g., P_1, P_38)
597 - Do NOT use letter suffixes like P_38a, P_38b - use P_38, P_39, P_40
598   instead
599 - Constraint IDs must be strictly C_N where N is a number
600
601 WEIGHTS
602 - 1.0: must, cannot, never, always, original modal statements
603 - 1.0: deontic "may"/"permitted"/"allowed" (permission in
604   contracts/legal)
605 - 1.0: factual statements, obligations, and permissions that are

```



```

606   clearly stated in the text
607   - 0.85: should, required
608   - 0.65: usually, typically, generally
609   - 0.40: sometimes, might, encouraged, epistemic "may" (possibility)
610   - 0.25: rarely, seldom
611
612   GRAMMAR
613   ¬ (not), (and), (or), (implies), (iff)
614
615   OUTPUT (JSON only)
616   {
617     "primitive_props": [
618       {
619         "id": "P_1",
620         "translation": "...",
621         "evidence": "Sentence N",
622         "explanation": "..."
623       }
624     ],
625     "constraints": [
626       {
627         "id": "C_1",
628         "formula": "P_1",
629         "translation": "...",
630         "llm_weight": 1.0,
631         "evidence": "...",
632         "reasoning": "..."
633       }
634     ]
635   }

```

Note, this exemplar has also been omitted for brevity.

A.2 Logification JSON format

The final knowledge base as a JSON file has three components:

```

639   {"primitive_props": [
640     {"id": "P_1", "translation": "...",
641      "evidence": "...", "explanation": "..."}],
642    "hard_constraints": [
643      {"id": "C_1", "formula": "P_1 P_2",
644       "translation": "...", "evidence": "...",
645       "reasoning": "...", "llm-weight": 1.0,
646       "nli_entailment": 1.0,
647       "combined=(llm)*(nli)": 1.0}],
648    "soft_constraints": [
649      {"id": "C_5", "formula": "P_6",
650       "translation": "...", "evidence": "...",
651       "reasoning": "...", "llm-weight": 0.65,
652       "nli_entailment": 1.0,
653       "combined=(llm)*(nli)": 0.65, "weight": 0.65}]]}

```

The id field refers to the index in the enumeration of the proposition or constraint; the translation field refers to its meaning in natural language; the evidence field refers to the first source location in the text where it appears; and the explanation field provides a brief reasoning for the LLM's choice. Note, while we retain the soft constraints in the final logified file, for the purposes of SAT solving they are ignored. We keep them for the possibility of extending the solvers to accommodate soft probabilistic logic.

A.3 Hypothesis Generation Prompt

For the DocNLI dataset, we regenerate queries using an LLM to obtain 4-way classification labels (entailment, contradiction, uncertain, not-mentioned) that better exercise logical rather than textual entailment. The prompt is below.

```

ROLE
You are a neuro-symbolic assistant translating natural language to
propositional logic.

TASK
Given text T and optional OpenIE triples which attempt to support T,
extract:
1. PRIMITIVE PROPOSITIONS (P_1, P_2, ...) - atomic, truth-evaluable
   statements
2. CONSTRAINTS (C_1, C_2, ...) - propositional formulas with
   llm_weight (0-1)

RULES
- Propositions must be simple declaratives that are atomic; they must
  not have connectives in them (e.g. they cannot be conditionals or be
  negations).
- Extract conditionals as separate propositions + implication
  constraint
- Modal words (may, should, typically, sometimes, rarely): create TWO
  propositions:
  * P_modal: original with modal word
  * P_base: without modal word
  Then add constraints: P_modal@1.0, P_base@weight,
  P_base==>P_modal@1.0

ID FORMAT (IMPORTANT)
- Proposition IDs must be strictly P_N where N is a number
  (e.g., P_1, P_38)
- Do NOT use letter suffixes like P_38a, P_38b - use P_38, P_39, P_40
  instead
- Constraint IDs must be strictly C_N where N is a number

WEIGHTS
- 1.0: must, cannot, never, always, original modal statements
- 1.0: deontic "may"/"permitted"/"allowed" (permission in
  contracts/legal)
- 1.0: factual statements, obligations, and permissions that are
  clearly stated in the text
- 0.85: should, required
- 0.65: usually, typically, generally
- 0.40: sometimes, might, encouraged, epistemic "may" (possibility)
- 0.25: rarely, seldom

GRAMMAR
¬ (not), (and), (or), (implies), (iff)

OUTPUT (JSON only)
{
  "primitive_props": [
    {
      "id": "P_1",
      "translation": "...",
      "evidence": "Sentence N",
      "explanation": "..."
    }
  ],
  "constraints": [
    {
      "id": "C_1",

```

```

723         "formula": "P_1",
724         "translation": "...",
725         "llm_weight": 1.0,
726         "evidence": "...",
727         "reasoning": "..."
728     }
729 ]
730 }

```

731 A.4 Query Translation Prompt

732 Given the top-10 retrieved propositions and a natural-language statement, we use the
733 prompt below on GPT-5 Nano to output a propositional formula or NONE. Note, the prompt
734 includes one exemplar but we have omitted it for brevity.

```

735 ROLE
736 You are a neuro-symbolic assistant translating natural language to
737 propositional logic.
738
739 TASK
740 Given text T and optional OpenIE triples which attempt to support T,
741 extract:
742 1. PRIMITIVE PROPOSITIONS (P_1, P_2, ...) - atomic, truth-evaluable
743    statements
744 2. CONSTRAINTS (C_1, C_2, ...) - propositional formulas with
745    llm_weight (0-1)
746
747 RULES
748 - Propositions must be simple declaratives that are atomic; they must
749   not have connectives in them (e.g. they cannot be conditionals or be
750   negations).
751 - Extract conditionals as separate propositions + implication
752   constraint
753 - Modal words (may, should, typically, sometimes, rarely): create TWO
754   propositions:
755   * P_modal: original with modal word
756   * P_base: without modal word
757   Then add constraints: P_modal@1.0, P_base@weight,
758   P_base==>P_modal@1.0
759
760 ID FORMAT (IMPORTANT)
761 - Proposition IDs must be strictly P_N where N is a number
762   (e.g., P_1, P_38)
763 - Do NOT use letter suffixes like P_38a, P_38b - use P_38, P_39, P_40
764   instead
765 - Constraint IDs must be strictly C_N where N is a number
766
767 WEIGHTS
768 - 1.0: must, cannot, never, always, original modal statements
769 - 1.0: deontic "may"/"permitted"/"allowed" (permission in
770   contracts/legal)
771 - 1.0: factual statements, obligations, and permissions that are
772   clearly stated in the text
773 - 0.85: should, required
774 - 0.65: usually, typically, generally
775 - 0.40: sometimes, might, encouraged, epistemic "may" (possibility)
776 - 0.25: rarely, seldom

```

777 A.5 Interesting case in LogicBench

778 An interesting use case for one reported failure from our pipeline arises in the following
779 LogicBench example: "If Mason decides to leave his job, he will not receive any salary." Our
780 pipeline is particularly strict about logical form and treats *deciding to leave* and *having left* as

distinct states, whereas the benchmark annotation appears to interpret the statement more pragmatically. Under this stricter notion of logical entailment, it is possible that Mason has decided to leave but has not yet acted on that decision, so his employment status is not definitively determined.

A.6 Results on other datasets

A.6.1 LogicBench

In this data set each query is either entailed or not entailed. In Table 3, for entailment examples (**Yes**), *Shortcuts*, *Expand*, and *Enrich* each improve accuracy by about +0.04, while *OpenIE* has a small negative effect. For not-entailment examples (**No**), *Enrich* has the largest negative effect, followed by *Expand* and *Shortcuts*, while *OpenIE* is near zero. The experimental design is valid (orthogonality checks pass) Effect directions and magnitudes with CIs "No significant main effects were detected, though T1 and T2 showed suggestive opposite effects on non-entailment questions (± 7.5)

We conducted a 2^{4-1}_{IV} fractional factorial experiment ($n=320$ questions) to assess four pipeline components: shortcut detection (Q_1), query expansion (Q_2), OpenIE extraction (T_1), and knowledge enrichment (T_2). No main effect reached statistical significance at $\alpha=0.05$. Effect sizes were small ($\approx$ accuracy $\pm 7.5\%$). Label-stratified analysis revealed suggestive but non-significant opposing effects of T_1 (+7.5

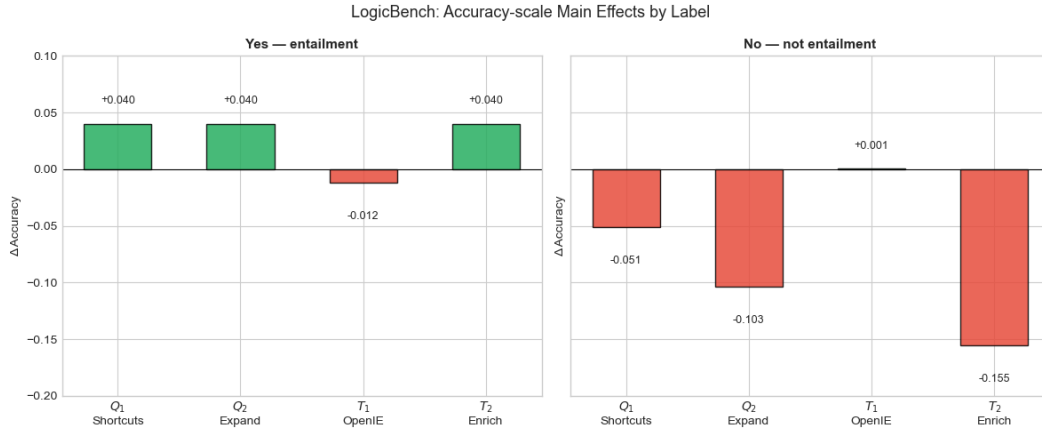


Figure 3: Main effects on accuracy for each factor in LogicBench, separated by label. Positive values indicate that switching a factor from its low level to its high level increases accuracy, while negative values indicate the opposite.